

ANÁLISIS Y DISEÑO DE ALGORITMOS I

PRÁCTICO N° 4. GREEDY

Construya algoritmos por la técnica Greedy para los siguientes problemas. Determine para cada uno su **complejidad temporal**.

1. **Problema de la Mochila.** Se tienen n objetos y una mochila. Para $i = 1, 2, \dots, n$, el objeto i tiene un peso positivo p_i y un valor positivo v_i . La mochila puede llevar un peso que no sobrepase P . El objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta. Los objetos pueden ser fraccionados, si una fracción x_i ($0 \leq x_i \leq 1$) del objeto i es ubicada en la mochila contribuye en $x_i p_i$ al peso total de la mochila y en $x_i v_i$ al valor de la carga. Formalmente, el problema puede ser establecido como:

$$\text{maximizar } \sum_{i=1}^n x_i v_i, \quad \text{con la restricción } \sum_{i=1}^n x_i p_i \leq P$$

donde $v_i > 0, p_i > 0$ y $0 \leq x_i \leq 1$ para $1 \leq i \leq n$.

Por ejemplo, para la instancia $n = 3$ y $P = 20$

$$(v_1, v_2, v_3) = (25, 24, 15)$$

$$(p_1, p_2, p_3) = (18, 15, 10)$$

Algunas soluciones posibles son:

(x_1, x_2, x_3)	$x_i p_i$	$x_i v_i$
$(1/2, 1/3, 1/4)$	16.5	24.25
$(1, 2/15, 0)$	20	28.2
$(0, 2/3, 1)$	20	31
$(0, 1, 1/2)$	20	31.5

puede observarse que $(0, 1, 1/2)$ produce el mayor beneficio.

2. **Planificación de tareas con plazo fijo.** Se deben procesar n tareas en un único procesador. Cada tarea se procesa en una unidad de tiempo y debe ser ejecutada en un plazo no superior a t_i . La tarea i produce una ganancia $g_i > 0$ si se procesa en un instante anterior a t_i . Una solución es factible si existe al menos una secuencia S de tareas que se ejecuten antes de sus respectivos plazos. Una solución óptima es aquella que maximiza la ganancia G tal que:

$$G = \sum_{s \in S} g_s$$

Diseñar un algoritmo para encontrar la solución óptima.

Por ejemplo, para la instancia $n = 4$ y los siguientes valores:

$$(g_1, g_2, g_3, g_4) = (50, 10, 15, 30)$$

$$(t_1, t_2, t_3, t_4) = (2, 1, 2, 1)$$

ANÁLISIS Y DISEÑO DE ALGORITMOS I

PRÁCTICO N° 4. GREEDY

Las planificaciones que hay que considerar y los beneficios correspondientes son:

Secuencia	Beneficio	Secuencia	Beneficio
1	50	2,1	60
2	10	2,3	25
3	15	3,1	65
4	30	4,1	80
1,3	65	4,3	45

Para maximizar el beneficio en este ejemplo, se debería ejecutar la secuencia 4,1.

3. **Minimizar tiempo de espera.** Un procesador debe atender n procesos. Se conoce de antemano el tiempo que necesita cada uno de ellos. Determinar en qué orden el procesador debe atender dichos procesos para minimizar la suma del tiempo que los procesos están en el sistema.

Por ejemplo, para $n = 3$ se tienen, los procesos (p_1, p_2, p_3) y tiempos de proceso (5, 10, 3)

Orden de atención	Tiempo de espera
p_1, p_2, p_3	$5 + (5+10) + (5+10+3) = 38$
p_1, p_3, p_2	$5 + (5+3) + (5+3+10) = 31$
p_2, p_1, p_3	$10 + (10+5) + (10+5+3) = 43$
p_2, p_3, p_1	$10 + (10+3) + (10+3+5) = 41$
p_3, p_1, p_2	$3 + (3+5) + (3+5+10) = 29$
p_3, p_2, p_1	$3 + (3+10) + (3+10+5) = 34$

La ordenación que produce el tiempo de espera mínimo es (p_3, p_1, p_2).

4. **Maximizar número de actividades compatibles.** Se tienen n actividades que necesitan utilizar un recurso, tal como una sala de conferencias, en exclusión mutua. Cada actividad i tiene asociado un tiempo de comienzo c_i y un tiempo de finalización f_i de utilización del recurso, con $c_i < f_i$. Si la actividad i es seleccionada se llevará a cabo durante el intervalo $[c_i, f_i)$. Las actividades i y j son compatibles si los intervalos $[c_i, f_i)$ y $[c_j, f_j)$ no se superponen (es decir, $c_i \geq f_j$ o $c_j \geq f_i$). El problema consiste en encontrar la cantidad máxima de actividades compatibles entre sí.